Input/Output

- **[execution time limit] 0.5 seconds (cpp)**
- **[memory limit] 1 GB**
- **[input] linkedlist.integer a**

  The first number, without its leading zeros.

  Guaranteed constraints:

  `0 ≤ a size ≤ 10^4`,

  `0 ≤ element value ≤ 9999`

- **[input] linkedlist.integer b**

  The second number, without its leading zeros.

  Guaranteed constraints:

  `0 ≤ b size ≤ 10^4`,

  `0 ≤ element value ≤ 9999`

- **[output] linkedlist.integer**

  The result of adding `a` and `b` together, returned without leading zeros in the same format.

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
    cout << "This prints to the console when you Run Tests
    return "Hello, " + name;
}
```

```
ListNode<int> * solution(ListNode<int> *, ListNode<int> * ) {

}
```

DESC

HISTORY

RULES

INFO

TTINGS

- [memory limit] 1 GB

- [input] array integer balances

  Array of integers, where `balances[i]` is the amount of money in the `(i + 1)th` account.

  Guaranteed constraints:

  `1 ≤ balances.length ≤ 100`

  `0 ≤ balances[i] ≤ 10^5`

- [input] array.string requests

  Array of requests in the order they should be processed. Each request is guaranteed to be in the format described above. It is guaranteed that requests come sequentially, i.e. the timestamp strictly increases.

  Guaranteed constraints:

  `1 ≤ requests.length ≤ 100`

- [output] array.integer

  The `balances` after executing all of the `requests` or array with a single integer - the index of the first invalid request preceded by `-`.

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
    cout << "This prints to the console when you Run Tests
    return "Hello, " + name;
}
```

main.cp

TO QUESTIONS

For `fractions = ["2/6+2/6", "7/10+13/10"]`, the output should be

`solution(fractions) = ["2/3", "2/1"]`

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] array.string fractions

  An array of strings, where each string contains an expression that represents the sum of two fractions, given in the form `"X/Y+U/V"`.

  Guaranteed constraints:

  `1 ≤ fractions.length ≤ 100`,

  `fractions[i]` has the form `"X/Y+U/V"` where `X, Y, U, V` are integers.

  `1 ≤ X, Y, U, V ≤ 2000`

- [output] array.string

  An array of strings, where the `i`th element is the result for the `i`th expression in the form `"A/B"`.

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
    cout << "This prints to the console when you Run Tests";
    return "Hello, " + name;
}
```

main.cpp

```
1  vec
2
3  }
4
```

Example 1

For `balances = [1000, 1500]` and `requests = ["withdraw 1613327630 2 480", "withdraw 1613327644 2 800", "withdraw 1614105244 1 100", "deposit 1614108844 2 200", "withdraw 1614108845 2 150"]`, the output should be `solution(balances, requests) = [900, 295]`.

Explanation

Here are the states of `balances` after each request:

- initially `[1000, 1500]`;
- `"withdraw 1613327630 2 480"` `[1000, 1020]`;
- `"withdraw 1613327644 2 800"` `[1000, 220]`;
- At `1613414030` the 2nd account will receive the cashback of `480 * 0.02 = 9.6`, which is rounded down to `9`: `[1000, 229]`;
- At `1613414044` the 2nd account will receive the cashback of `800 * 0.02 = 16`: `[1000, 245]`;
- `"withdraw 1614105244 1 100"` `[900, 245]`;
- `"deposit 1614108844 2 200"` `[900, 445]`;
- `"withdraw 1614108845 2 150"` `[900, 295]`, which is the answer.
- Cashbacks for the last two withdrawals happen at `1614191644` and `1614195245`, which is after the last request timestamp.

Example

- For `a = [9876, 5432, 1999]` and `b = [1, 8001]`, the output should be

  `solution(a, b) = [9876, 5434, 0]`

  Explanation: `9876543321999 + 18001 = 9876543340000`

- For `a = [123, 4, 5]` and `b = [100, 100, 100]`, the output should be

  `solution(a, b) = [223, 104, 105]`

  Explanation: `12300040005 + 10001000100 = 22301040105`

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] linkedlist.integer a

  The first number, without its leading zeros.

  Guaranteed constraints:

  $0 \leq a$ size $\leq 10^4$,

  $0 \leq$ element value ≤ 9999.

- For `crypt = ["SEND", "MORE", "MONEY"]`, the output should be

  `solution(crypt) = 1`, because there is only one solution to this *cryptarithm*:

  `O = 0`, `M = 1`, `Y = 2`, `E = 5`, `N = 6`, `D = 7`, `R = 8`, and `S = 9` (`9567 + 1085 = 10652`).
- For `crypt = ["GREEN", "BLUE", "BLACK"]`, the output should be

  `solution(crypt) = 12`, because there are `12` possible valid solutions:
  - `54889 + 6138 = 61027`
  - `18559 + 2075 = 20634`
  - `72449 + 8064 = 80513`
  - `48229 + 5372 = 53601`
  - `47119 + 5261 = 52380`
  - `36887 + 4028 = 40915`
  - `83447 + 9204 = 92651`
  - `74665 + 8236 = 82901`
  - `66884 + 7308 = 73192`

Example 2

For `balances = [20, 1000, 500, 40, 90]` and `requests = ["deposit 1633327630 1 400", "withdraw 1633327635 1 20", "withdraw 1633327651 1 50", "deposit 1633327655 1 50"]`, the output should be `solution(balances, requests) = [-1]`.

Explanation

Here are the states of `balances` after each request:

- initially `[20, 1000, 500, 40, 90]`
- `"deposit 1633327630 1 400"` `[20, 1000, 900, 40, 90]`
- `"withdraw 1633327635 1 20"` `[0, 1000, 900, 40, 90]`
- `"withdraw 1633327651 1 50"` it is not possible to withdraw `50` from the 1st account, so the request is invalid
- the rest of the requests are not processed

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] array.integer balances

  Array of integers, where `balances[i]` is the amount of money in the $(i + 1)^{th}$ account.

  Guaranteed constraints:

  $1 \leq$ `balances.length` ≤ 100.

You've been asked to program a bot for a popular bank that will automate the management of incoming requests. Every request has its own `timestamp` *in seconds*, and it is guaranteed that all requests come sequentially, i.e. the timestamp is strictly increasing. There are two types of incoming requests:

- `deposit <timestamp> <holder_id> <amount>` — request to deposit `<amount>` amount of money in the `<holder_id>` account;
- `withdraw <timestamp> <holder_id> <amount>` — request to withdraw `<amount>` amount of money from the `<holder_id>` account. As a bonus, bank also provides a cashback policy — `2%` of the withdrawn amount rounded down to the nearest integer will be returned to the account exactly 24 hours after the request timestamp. If the cashbask and deposit/withdrawal happen at the same timestamp, assume cashback happens earlier.

Your system should also handle invalid requests. There are two types of invalid requests:

Your task is to find the sum of two fractions, expressed as the form `x/y` and `u/v`, where `x`, `y`, `u` and `v` are four integers. Compute their sum and reduce it to its lowest indivisible state: `A/B`.

For example:

- `2/6+2/6` equals `4/6`, which should be reduced to `2/3`.
- `7/10+11/10` equals `18/10` which should be reduced to `9/5`.

You are given an array of strings, which contains several expressions in the form `"x/y+u/v"`. Return a string array, where the i^{th} element is the result for the i^{th} expression in the form `"A/B"`.

Example

For `fractions = ["2/6+2/6", "7/10+11/10"]`, the output should be

`solution(fractions) = ["2/3", "9/5"]`.

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] array.string fractions

  An array of strings, where each string contains an expression that represents the sum of two fractions, given in the form `"x/y+u/v"`.

  Guaranteed constraints:

  `0 ≤ fractions.length ≤ 100`,

  `fractions[i]` contains only `"x/y+u/v"` strings, `x`, `y`, `u` and `v` are integers.

Your system should also handle invalid requests. There are two types of invalid requests:

- invalid account number;
- withdrawal of a larger amount of money than is currently available.

For the given list of initial `balances` and `requests`, return the state of `balances` straight after the last request has been processed, or an array of a single element `[-<request_id>]` (please note the minus sign), where `<request_id>` is the 1-based index of the first invalid request. Note that cashback requests which haven't happened before the last request should be ignored.

Example

Example 1

Input/Output

- **[execution time limit] 0.5 seconds (cpp)**
- **[memory limit] 1 GB**
- **[input] array.string crypt**

  Array of three non-empty strings containing only uppercase English letters.

  Guaranteed constraints:

  `1 ≤ crypt[i].length ≤ 35`

- **[output] integer**

  The number of valid solutions.

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
    cout << "This prints to the console when you Run Tests
    return "Hello, " + name;
}
```

A *cryptarithm* is a mathematical puzzle where the goal is to find the correspondence between letters and digits such that the given arithmetic equation consisting of letters holds true.

Given a *cryptarithm* as an array of strings `crypt`, count the number of its valid solutions.

The solution is valid if each letter represents a different digit, and the leading digit of any multi-digit number is not zero.

`crypt` has the following structure `[word1, word2, word3]`, which stands for the `word1 + word2 = word3` *cryptarithm*.

Example

- For `crypt = ["SEND", "MORE", "MONEY"]`, the output should be

  `solution(crypt) = 1`, because there is only one solution to this *cryptarithm*:

  `O = 0`, `M = 1`, `Y = 2`, `E = 5`, `N = 6`, `D = 7`, `R = 8`, and `S = 9` (`9567 + 1085 = 10652`).
- For `crypt = ["GREEN", "BLUE", "BLACK"]`, the output should be

  `solution(crypt) = 12`, because there are `12` possible valid solutions:
  - `54889 + 6138 = 61027`

be

```
solution(fractions) = ["2/3", "2/1"]
```

Input/Output

- [execution time limit] 0.5 seconds (cpp)
- [memory limit] 1 GB
- [input] array.string fractions

  An array of strings, where each string contains an expression that represents the sum of two fractions, given in the form "X/Y+U/V"

  Guaranteed constraints:

  1 ≤ fractions.length ≤ 500,

  fractions[i] has the form "X/Y+U/V" where X, Y, U, V are integers.

  1 ≤ X, Y, U, V < 2000

- [output] array.string

  An array of strings, where the i**-th element is the result for the i**-th expression in the form "A/B"

[C++] Syntax Tips

```
// Prints help message to the console
// Returns a string
string helloWorld(string name) {
    cout << "This prints to the console when you Run Code";
    return "Hello, " + name;
}
```

A *cryptarithm* is a mathematical puzzle where the goal is to find the correspondence between letters and digits such that the given arithmetic equation consisting of letters holds true.

Given a *cryptarithm* as an array of strings `crypt`, count the number of its valid solutions.

The solution is valid if each letter represents a different digit, and the leading digit of any multi-digit number is not zero.

`crypt` has the following structure: `[word1, word2, word3]`, which stands for the `word1 + word2 = word3` *cryptarithm*.

Example

- For `crypt = ["SEND", "MORE", "MONEY"]`, the output should be

  `solution(crypt) = 1`, because there is only one solution to this *cryptarithm*:

  `O = 0`, `M = 1`, `Y = 2`, `E = 5`, `N = 6`, `D = 7`, `R = 8`, and `S = 9` (`9567 + 1085 = 10652`)
- For `crypt = ["GREEN", "BLUE", "BLACK"]`, the output should be

  `solution(crypt) = 12`, because there are `12` possible valid solutions:
  - `54889 + 6138 = 61027`